

JavaScript Problem Solving Roadmap

A structured progression of 10 essential code challenges from fundamental syntax to advanced programming patterns.

● Easy Level: Warm-ups & Core Syntax

Task 01 Reverse a String

Write a function `reverseString(str)` that takes a string input and returns that exact string flipped backwards.

Concepts Covered	String manipulation, array conversions (<code>split</code> , <code>reverse</code> , <code>join</code>), or iterative loops.
Example Case	<code>reverseString("hello")</code> should return <code>"olleh"</code>

Task 02 FizzBuzz Scenario

Write a function `fizzBuzz(n)` that prints or returns numbers from 1 up to `n` . For multiples of 3, use `"Fizz"` instead of the number. For multiples of 5, use `"Buzz"` . For multiples of both, use `"FizzBuzz"` .

Concepts Covered	Modulo operators (<code>%</code>), conditional branching (<code>if/else</code>), loop control structures.
Example Case	<code>fizzBuzz(15)</code> prints numbers with 3rd, 5th, and 15th slots replaced by corresponding terms.

Task 03 Find the Largest Number

Write a function `findMax(arr)` that receives an array of numbers and extracts the highest single numerical value.

Concepts Covered	Array iteration, accumulators, ES6 Spread Operator (<code>...</code>), or <code>Math.max()</code> built-ins.
Example Case	<code>findMax([1, 5, 8, 3])</code> should return <code>8</code>

Basic to Intermediate Level: Data Structures & Logic

Task 04 Count Vowels

Write a function `countVowels(str)` that scans a text string and outputs the combined occurrence count of vowels (a, e, i, o, u), ignoring capitalization.

Concepts Covered	String searches, collections/lookup setups, or Regular Expressions (<code>/[aeiou]/gi</code>).
Example Case	<code>countVowels("javascript")</code> should return <code>3</code>

Task 05 Remove Duplicates from Array

Write a function `removeDuplicates(arr)` that takes an array and filters out any repeating values, returning a clean array with unique values only.

Concepts Covered	Higher-order functions (<code>filter</code> , <code>indexOf</code>), or modern data structures like a JavaScript <code>Set</code> .
Example Case	<code>removeDuplicates([1, 2, 2, 3, 4, 4])</code> should return <code>[1, 2, 3, 4]</code>

Task 06 Check for Palindrome

Write a function `isPalindrome(str)` that checks whether a phrase reads identically both forwards and backwards, discounting external spaces, punctuation, or uneven casing.

Concepts Covered	Sanitization string formatting, reverse comparisons, two-pointer parsing loops.
Example Case	<code>isPalindrome("racecar")</code> should return <code>true</code>

Task 07 Title Case a Sentence

Write a function `titleCase(str)` that modifies every word within a string sentence to begin with an upper-case character while leaving following letters lowercased.

Concepts Covered	String slicing (<code>substring</code> , <code>slice</code>), array mapping mechanisms, index tracking.
Example Case	<code>titleCase("i love coding")</code> should return <code>"I Love Coding"</code>

Advanced Level: Closures, Async, & Algorithms

Task 08 Two Sum Algorithm

Given an array of integer elements `nums` along with a target sum integer `target`, look up and return the exact element *indices* of the pair that yields the target sum value.

Concepts Covered	Time and space complexity minimization. Optimizing from brute-force $O(n^2)$ down to linear $O(n)$ via Hash Map collections (JavaScript <code>Map</code> or object literals).
Example Case	<code>twoSum([2, 7, 11, 15], 9)</code> should return <code>[0, 1]</code> because $2 + 7 = 9$.

Task 09 Memoized Function Decorator

Write a higher-order wrapper function `memoize(fn)` that accepts an expensive calculation sequence and returns a optimized proxy. If execution matches previous parameters, return the saved cache reference instead of computing again.

Concepts Covered	Functional patterns, execution Closures, Rest/Spread arguments passing, value-key serialization.
Example Case	Repeatedly querying a memoized factorial callback immediately returns execution from local lookup definitions.

Task 10 Asynchronous Fetch Timeout wrapper

Construct a utility function `fetchWithTimeout(url, ms)` that executes a traditional network request via `fetch()`. If the remote service fails to resolve before `ms` time passes, automatically trigger a Promise failure statement throwing a `"Request Timed Out"` error.

Concepts Covered	Asynchronous flow orchestration, JavaScript <code>Promise</code> APIs, <code>Promise.race()</code> structures, and timer control loops (<code>setTimeout</code>).
Example Case	Setting a low millisecond count like <code>fetchWithTimeout('https://api.com', 200)</code> safely catches hanging requests.